QLoRA Distillation for Algorithmic Code Classification and Generation

Addressing the "Valley of Code Reasoning" via Decoupled Algorithmic Conditioning

Pranav Bhat P (231IT049) **Shivam Kumar** (231IT068) **Himanshu Mehar** (231IT027)

Course: IT488 – Large Language Models
Department of Information Technology
National Institute of Technology Karnataka (NITK), Surathkal

Midsemester Evaluation – March 2026

Presentation Outline

Base Paper: Findings & Drawbacks (He et al., 2025)

Key Findings of Base Paper

- **The "Valley of Code Reasoning"**: Intermediate data scaling exhibits a sharp performance dip where student reasoning degrades before recovering.
- **Entangled Token Generation**: Jointly predicting rationale and code forces the model to resolve abstract math and low-level syntax simultaneously.
- **CoT Fragility in Code**: Standard Chain-of-Thought prompts frequently trigger premature convergence to suboptimal brute-force loops ($O(N^2)$).

Critical Drawbacks of Base Paper

- **No Decoupled Planning**: Lacks an explicit System-2 stage to identify the algorithmic paradigm (DP, Graphs, Greedy) before code synthesis.
- **Unfiltered Teacher Contamination**: Assumes teacher rationales are always sound; hallucinated or mismatched traces pollute student supervision.
- **High Compute Dependency**: Evaluated only on multi-billion parameter models ($\geq$ 7B), lacking accessible QLoRA solutions for sub-1B models on consumer GPUs.

Our Approach to Address These Drawbacks

1. Decoupled Algorithmic Conditioning (Stage 1 Classifier) + **2. Semantic Rationale Filter** (purging noise) + **3. Consumer-Grade QLoRA** (Qwen2.5-0.5B on single 15GB T4 GPU).

Problem Statement

Core Vulnerabilities in Existing Code Distillation

1. **Unstructured Monolithic CoT Generation**: Student models attempt to predict algorithmic intuition and raw syntax simultaneously, leading to hallucinated loops and asymptotic timeout errors ($O(N^2)$ instead of $O(N \log N)$).
2. **Teacher Trace Contamination**: Commercial teacher LLMs generate deceptive or mislabeled rationales that diverge from ground-truth paradigms. Distilling noisy traces degrades student reasoning.
3. **High Compute Barriers**: Full parameter fine-tuning of multi-billion parameter models is inaccessible to standard edge and consumer hardware.

Project Objectives

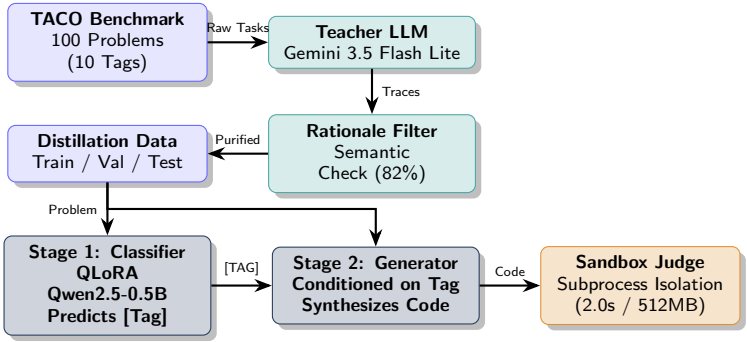
Midsemester Scope (40%) – Completed

- **100-Problem Benchmark**: Stratified extraction from TACO across 10 algorithmic categories.
- **Teacher Trace Synthesis**: Structured rationales from Gemini 3.5 Flash Lite.
- **Rationale-Consistency Filter**: Automatic semantic verification (82% retention).
- **Subprocess Sandbox Judge**: Timeout (2.0s) and memory (512MB) isolation.
- **Stage 1 QLoRA Distillation**: Comparative training of HF PEFT vs. Unsloth.

Endsemester Scope (60%) – In Progress

- **Stage 2 Conditioned Generator**: Conditioning on [TAG] + [PROBLEM] to synthesize [RATIONALE] + [CODE].
- **Empirical Scaling Sweep**: Investigating the "Valley of Code Reasoning" across 10%, 25%, 50%, 100%.
- **Full Pass@k Benchmark**: Sandbox execution judging across all problem suites.
- **Ablation Study**: Conditioned vs. Unconditioned baseline comparison.

Methodology: Decoupled Two-Stage Distillation



Two-Stage Separation Principle

Stage 1 (System 2 Planning): Predicts the algorithmic paradigm (e.g., Dynamic Programming, Graph BFS/DFS).

Stage 2 (Execution & Synthesis): Conditioned on the committed paradigm, generates verified rationale and syntax.

Quality Assurance: Rationale Filter & Sandbox Judge

Semantic Rationale-Consistency Filter

- Validates semantic alignment between stated teacher strategy and ground-truth taxonomy.
- Eliminates length-deficient traces (< 40 characters) and cross-paradigm contradictions.
- Formula:

$$\text{Score} = \min(1.0, 0.4 + 0.2 \cdot |\mathcal{K}_{\text{matched}}|)$$

- **Empirical Yield**: 82/100 retained (82.0% retention yield, 18 noisy traces purged).

Sandboxed Subprocess Execution Judge

- Enforces strict automated evaluation in decoupled child processes.
- **POSIX Limits**: `RLIMIT_AS` restricts memory to 512MB.
- **Timeout Protection**: `subprocess.TimeoutExpired` capped at 2.0s.
- Returns standardized verdicts:

AC (Accepted) **WA** (Wrong Answer)
TLE (Time Limit) **RE** (Runtime Error)

Implementation: 10-Class Algorithmic Taxonomy

#	Algorithmic Category	Problems	Representative Problem Strategy
1	Dynamic Programming	10	Subset sum, knapsack, state-transitions
2	Greedy Algorithms	10	Interval scheduling, priority choices
3	Graph Theory	10	BFS, DFS, Dijkstra shortest path
4	Mathematics	10	Combinatorics, parity invariants, geometry
5	Data Structures	10	Heaps, monotonic stacks, DSU, priority queues
6	Tree Algorithms	10	Binary trees, subtree traversal, LCA
7	Brute Force / Simulation	10	Systematic state enumeration, grid crawl
8	String Manipulation	10	Palindromes, prefix functions, parsing
9	Number Theory	10	GCD/LCM, modular arithmetic, prime sieves
10	Binary Search	10	Monotonic predicate search, bisect space

Table: Curated 100-Problem TACO Benchmark Distribution (800–1200 Rating)

Implementation: Dual QLoRA Distillation Pipelines

Pipeline A: Hugging Face PEFT

- **Model**: `AutoModelForSequenceClassification` on `Qwen/Qwen2.5-0.5B`.
- **Quantization**: BitsAndBytes 4-bit NormalFloat (`NF4`) with double-quantization.
- **LoRA Configuration**:
  - Rank $r = 16$, Alpha $\alpha = 32$
  - Targets: `q_proj`, `k_proj`, `v_proj`, `o_proj`
  - Dropout: 0.05
- **Trainable Parameters**: 2.17M (0.44% of 496M).

Pipeline B: Unsloth FastLanguageModel

- **Model**: `FastLanguageModel` with Triton kernel acceleration.
- **Optimization**: Direct backpropagation into custom OpenAI Triton cross-entropy / LoRA kernels.
- **Linear Projection**:
  - Feature Dimension: $151,936$ (vocab projection)
  - Output Dimension: 10 classes
  - Checkpointing: `"unsloth"`
- **Isolation**: Two independent Colab notebooks created to eliminate kernel monkey-patch collisions.

Experimental Results: Benchmark Comparison

Evaluation Metric	Pipeline A (HF PEFT)	Pipeline B (Unsloth)
Base Student LLM	Qwen2.5-0.5B	Qwen2.5-0.5B
Quantization Precision	4-bit NF4	4-bit NF4 (Triton)
Trainable Parameters	2.17M (0.44%)	2.17M (0.44%)
Total Training Time (4 Epochs)	**65.01 s**	**21.55 s (3.0× Faster)**
Peak VRAM Memory Allocated	**992.6 MB**	**971.1 MB**
Top-1 Classification Accuracy	**17.65%**	**5.88%**
Top-3 Classification Accuracy	**41.18%**	**23.53%**
Macro F1-Score	**0.0952**	**0.0222**
Training Loss Trajectory	$7.21 \rightarrow 0.33$	$2.31 \rightarrow 0.41$

Table: Empirical Benchmark on Google Colab (Tesla T4 GPU, 17 Unseen Test Problems)

Diagnostic Insights: Why Are Initial Accuracies Modest?

Scientific Root-Cause Analysis

1. **Severe Sample Sparsity**:
   - Training on 58 problems across 10 categories yields only $\sim$ 5–6 problems per class.
   - Loss plummeting from 7.21 to 0.33 indicates **rapid sample memorization** rather than general algorithmic reasoning.
2. **The Random Linear Head Penalty on Causal LMs**:
   - `Qwen2.5` is an autoregressive generative model. Slapping an uninitialized linear head discards pre-trained generative semantics.
   - For Unsloth, learning $151,936 \times 10 = 1.52$M fresh weights from 58 samples is mathematically underdetermined.
3. **Context Truncation at 256 Tokens**:
   - Standard competitive programming problems span 400–800 tokens.
   - Truncation cuts off constraints and edge-case specs where algorithmic signatures reside.

Proposed System Improvements (Closing the Gap)

1. Generative Instruction Tuning

- Replace discrete sequence classification with native causal token prediction.
- Prompt format: `"Classify into [DP, Graphs, ...]: Problem: ... Category: Graphs"`
- Directly exploits pre-trained token embeddings without learning fresh linear layers.

2. Context Length Expansion

- Increase context window from 256 $\rightarrow$ 512 tokens.
- Retains full problem statement and constraint blocks within T4 VRAM budget ($<$ 1.5GB).

3. Full Linear Layer LoRA Adapters

- Target both attention and MLP feedforward blocks: `[q, k, v, o, gate, up, down_proj]`.
- Captures algorithmic knowledge stored in transformer MLP projections.

4. Data Scaling to 250+ Problems

- Expand dataset to 25–50 problems per category.
- Reaches empirical few-shot threshold for stable LoRA generalization.

Future Work

Key Deliverables for Endsemester:

1. **Stage 2 Conditioned Generator**:

   $$[\text{TAG}]+[\text{PROBLEM}] \longrightarrow [\text{RATIONALE}]+[\text{CODE}]$$

   Synthesizes verified Python 3 solutions.

2. **Unconditioned Baseline Model**:

   $$[\text{PROBLEM}] \longrightarrow [\text{CODE}]$$

   Serves as rigorous ablation baseline.

3. **"Valley of Code Reasoning" Scaling Sweep**: Empirical evaluation across 10%, 25%, 50%, and 100% training data.

Evaluation Benchmark: Pass@k

- **Metric**: Strict unit-test execution pass rate:

$$\text{Pass@}k = \mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

- Executed via the Subprocess Sandbox Judge under 2.0s CPU / 512MB RAM limits.
- Measures functional code correctness over $k \in \{1, 5, 10\}$ sample generations per problem.

Summary of Implemented Work

Midsemester Milestone Achievements (40% Scope Complete)

- ✓ **Dataset Pipeline**: Curated 100-problem TACO benchmark with Gemini 3.5 Flash Lite teacher traces across 10 algorithmic classes.
- ✓ **Quality Control**: Semantic Rationale-Consistency Filter achieves 82% retention yield.
- ✓ **Sandbox Judge**: Subprocess isolation enforces 2.0s timeout and 512MB RAM limits.
- ✓ **Dual QLoRA Framework**: Empirically compared Hugging Face PEFT vs. Unsloth ($3\times$ faster throughput, $<$ 1GB VRAM).
- ✓ **Colab Notebooks**: Two standalone, self-contained reproducible environments generated and validated.

Thank You!

Questions & Discussion